ACADEMIC PROJECT DOCUMENTATION & FUNCTION FLOWCHARTS

Python Activity-Based Learning — 10 Case Studies

This repository contains Python solutions for ten case studies selected from the **Python Activity-Based Learning** assignment. The programs are designed using beginner-level Python concepts such as variables, data types, operators, conditional statements, loops, strings, lists, nested lists, tuples, and tuple unpacking.

Student Details

Field	Details
Student Name	_____
Roll Number	_____
Registration Number	_____
Course / Branch	B.Tech -- CSE (AI & ML)
Year / Semester	_____
Section	_____
Subject	Python Programming
Faculty Name	_____
University / College	_____
Academic Year	_____
GitHub Repository Link	_____

Project Objectives

- Apply Python programming concepts to practical, real-world problems.
- Develop logical thinking through algorithms, visual flowcharts, and structured pseudocode.
- Understand how to take input, process data, and display meaningful output.
- Test programs thoroughly using normal, boundary, and invalid input values.
- Organize and document Python programs cleanly in a GitHub repository.

> ***Case Studies Included:*** *The following pages present the first ten case studies from the assignment document alongside their respective function execution flowcharts. If a different set of ten was selected for the Python programs, update this list to match those filenames and topics.*

1. University Scholarship Allocation

`def evaluate_scholarship_applicants()`

Description: Evaluates scholarship applicants using academic marks, subject-wise minimum scores, attendance, and family income criteria. **Objective:** Identify eligible students, rank them, and resolve ties using predefined rules.

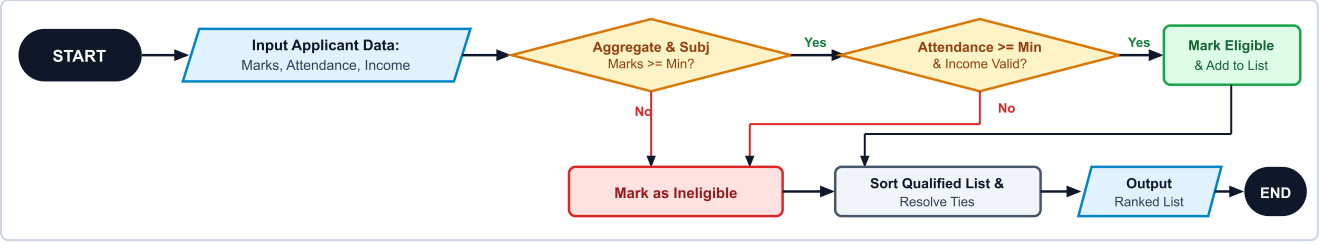
Inputs: Student name/ID, aggregate marks, subject marks, attendance %, family income.

Processing: Check each eligibility condition, select qualified applicants, compare scores.

Expected Output: Eligibility status and a ranked list of scholarship applicants.

Python Concepts: Variables, input, arithmetic operators, conditionals, lists, sorting logic.

FUNCTION FLOWCHART — EVALUATE_SCHOLARSHIP_APPLICANTS()



2. Progressive Electricity Billing

`def calculate_electricity_bill()`

Description: Calculates household electricity bills using different consumption slabs. **Objective:** Find the total bill after applying fixed charges, surcharges, and rebates.

Inputs: Units consumed, slab rates, fixed charge, surcharge, rebate details.

Processing: Calculate charge for each applicable slab and add/subtract relevant charges.

Expected Output: Unit-wise charge details and the final electricity bill.

Python Concepts: Variables, arithmetic operators, if-elif-else, conditional calculations.

Execution Flow (`calculate_electricity_bill`): [1. Start] → **[2. Input]** Read units consumed, slab tariff rates, fixed charge, surcharge rules, and rebate threshold → **[3. Slab Branching]** Evaluate `if-elif-else` consumption tiers (Slab 1, Slab 2, Slab 3+) and compute progressive energy charges → **[4. Process]** Add fixed meter charge → **[5. Condition]** If consumption exceeds surcharge limit, add surcharge; if units qualify for low-usage rebate, subtract rebate → **[6. Output & End]** Display slab-wise breakdown and final payable bill.

3. Hospital Billing and Patient Classification

`def generate_hospital_bill()`

Description: Calculates hospital bills from consultation, diagnostic, and treatment charges. **Objective:** Generate a bill and classify patients according to a given payment policy.

Inputs: Consultation fee, diagnostic charges, treatment charges, concession details.

Processing: Add charges, validate bill, apply eligible concessions, determine payment category.

Expected Output: Itemized hospital bill, concession amount, final payable amount, patient category.

Python Concepts: Variables, arithmetic operators, conditional statements, formatted output.

Execution Flow (`generate_hospital_bill`): [1. Start] → **[2. Input & Validate]** Read consultation, diagnostic, and treatment fees; verify all values are non-negative → **[3. Process]** Compute `gross_bill = consultation + diagnostic + treatment` → **[4. Decision]** Check patient concession eligibility (senior citizen/insurance/scheme) and deduct applicable discount → **[5. Classification]** Assign payment category (Standard, Subsidized, Priority/Critical Care) based on final bill policy → **[6. Output & End]** Print itemized invoice and classification.

4. Employee Payroll and Deduction Analysis

`def calculate_employee_payroll()`

Description: Calculates employee salary using basic pay, allowances, overtime, and incentives. **Objective:** Determine gross salary, deductions, and final take-home salary.

Inputs: Basic pay, allowances, overtime amount, incentives, deduction details.

Processing: Calculate gross salary, apply specified deductions, and find net salary.

Expected Output: Formatted salary statement showing earnings, deductions, and net salary.

Python Concepts: Variables, arithmetic operators, input/output, conditional statements.

Execution Flow (`calculate_employee_payroll`): [1. Start] → **[2. Input]** Read basic pay, HRA/DA allowances, overtime hours/pay, performance incentives, and tax/PF rates → **[3. Earnings Process]** Compute `gross_salary = basic + allowances + overtime + incentives` → **[4. Deduction Process]** Evaluate conditional tax slab, Provident Fund (PF), and professional tax to compute `total_deductions` → **[5. Net Pay]** Calculate `net_salary = gross_salary - total_deductions` → **[6. Output & End]** Display formatted payslip.

5. Multi-Plan Telecommunications Cost Optimization

`def optimize_telecom_plan()`

Description: Compares mobile plans based on usage, validity, and budget. **Objective:** Identify the most economical plan that satisfies the user's requirements.

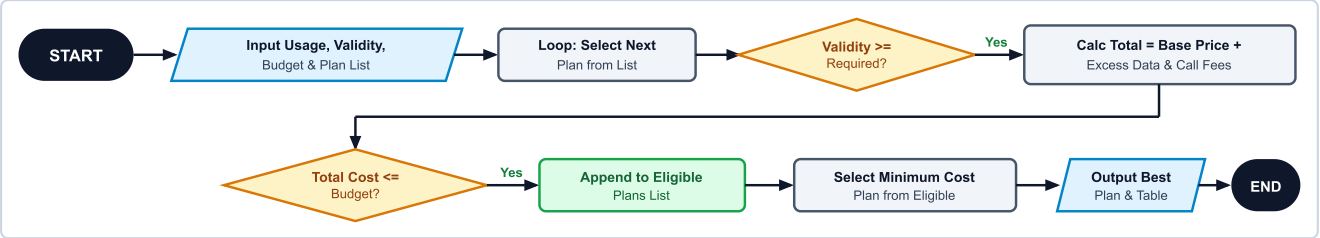
Inputs: Data usage, call usage, plan price, validity, and budget.

Processing: Check plan conditions, calculate excess charges where applicable, compare eligible plans.

Expected Output: Plan-wise cost comparison and the most economical eligible plan.

Python Concepts: Variables, arithmetic operators, if-elif-else, loops, and lists.

FUNCTION FLOWCHART — OPTIMIZE_TELECOM_PLAN()



6. Railway Fare and Reservation Calculator

`def calculate_railway_fare()`

Description: Calculates railway fares for multiple passengers using distance, travel class, age concessions, and reservation fees. **Objective:** Find total journey fare and apply cancellation penalty when required.

Inputs: Passenger details, distance, travel class, age, reservation fee, cancellation details.

Processing: Calculate base fare, apply concessions, add reservation fees, calculate penalties.

Expected Output: Passenger-wise fare details and the total reservation amount.

Python Concepts: Variables, arithmetic operators, conditional statements, loops, and lists.

Execution Flow (`calculate_railway_fare`): [1. Start] → **[2. Input]** Read journey distance, class multiplier, reservation fee, and passenger list → **[3. Loop Passengers]** Compute base fare; if age qualifies (child/senior), deduct concession; add reservation fee and accumulate total → **[4. Cancellation Check]** If ticket cancelled, deduct slab penalty → **[5. Output & End]** Display passenger breakdown and net total.

7. Retail Billing with Multiple Discount Rules

`def generate_retail_invoice()`

Description: Generates a retail bill for multiple products and quantities. **Objective:** Calculate the final invoice after applying product taxes and discount rules.

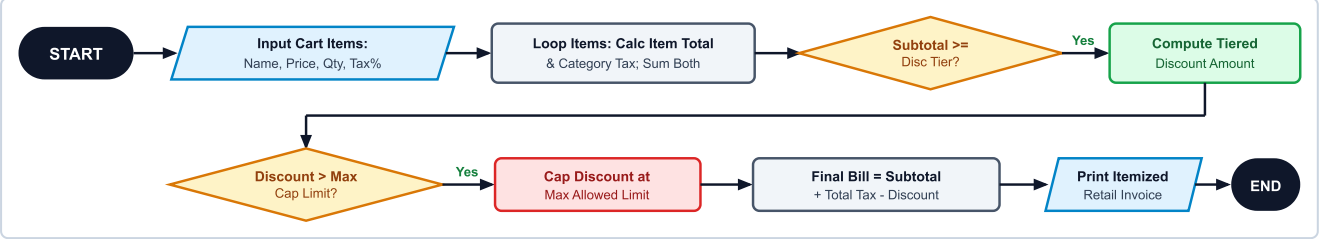
Inputs: Product names, prices, quantities, tax rates, and discount conditions.

Processing: Calculate item totals, apply category taxes, check discount thresholds, cap max concession.

Expected Output: Itemized invoice with subtotal, tax, discount, and final payable amount.

Python Concepts: Strings, lists, loops, arithmetic operators, and conditional statements.

FUNCTION FLOWCHART — GENERATE_RETAIL_INVOICE()



8. Manufacturing Cost and Profitability Analysis

`def analyze_manufacturing_profitability()`

Description: Calculates manufacturing expenses, including materials, labour, wastage, and machine costs. **Objective:** Determine whether production meets a specified profit target.

Inputs: Material, labour, wastage, machine costs, production quantity, selling price.

Processing: Calculate total production cost, cost per unit, revenue, and profit.

Expected Output: Cost summary, expected revenue, profit or loss, and target status.

Python Concepts: Variables, arithmetic operators, conditional statements, and loops.

Execution Flow (`analyze_manufacturing_profitability`): [1. Start] → **[2. Input]** Read material, labour, wastage, machine costs, units produced, selling price, and target profit → **[3. Process]** Compute `total_cost`, `unit_cost = total_cost / qty`, `revenue = qty * price`, and `net_profit = revenue - total_cost` → **[4. Decision]** Evaluate whether `net_profit >= target_profit` (Target Met), `0 <= net_profit < target` (Below Target), or `net_profit < 0` (Loss) → **[5. Output & End]** Display profitability summary.

9. Multi-Currency Travel Budget Analysis

`def analyze_travel_budget()`

Description: Converts travel expenses from different currencies using supplied exchange rates. **Objective:** Compare total planned spending with travel budget and identify largest expense category.

Inputs: Expense amounts, currency exchange rates, conversion fees, and total budget.

Processing: Convert expenses into base currency, add conversion fees, sum categories, compare with budget.

Expected Output: Converted expense summary, largest expense category, and budget status.

Python Concepts: Variables, arithmetic operators, lists, loops, and conditional statements.

Execution Flow (`analyze_travel_budget`): [1. Start] → **[2. Input]** Read total base-currency budget and list of foreign expense records → **[3. Loop Expenses]** Convert foreign amount using exchange rate, add forex fee, accumulate category totals → **[4. Max & Budget Check]** Identify largest expense category and compare total spending against budget → **[5. Output & End]** Display summary and surplus/deficit.

10. Campus Event Financial Planning

`def plan_campus_event_finances()`

Description: Calculates the cost of organizing a campus event. **Objective:** Determine participant contributions, sponsorship shortfall, and minimum registrations needed to break even.

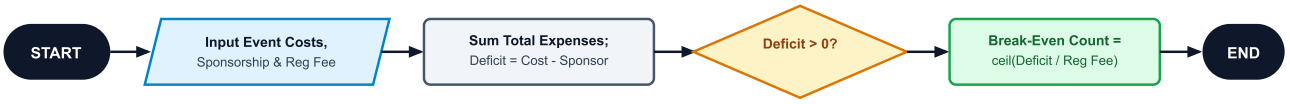
Inputs: Venue, equipment, catering, transport, certificate costs, sponsorship, registration fee.

Processing: Calculate total expenses, subtract sponsorship, estimate participant collection, find break-even count.

Expected Output: Event budget, amount to be collected, sponsorship shortfall, break-even registrations.

Python Concepts: Variables, arithmetic operators, conditional statements, and loops.

FUNCTION FLOWCHART — PLAN_CAMPUS_EVENT_FINANCES()



Repository Structure

```
Python-Activity-Based-Learning/
├── README.md
├── Case_01_University_Scholarship.py
├── Case_02_Electricity_Billing.py
├── Case_03_Hospital_Billing.py
├── Case_04_Employee_Payroll.py
├── Case_05_Telecom_Plan_Optimization.py
├── Case_06_Railway_Fare_Calculator.py
├── Case_07_Retail_Billing.py
├── Case_08_Manufacturing_Profitability.py
├── Case_09_Travel_Budget_Analysis.py
├── Case_10_Campus_Event_Planning.py
└── screenshots/
    ├── Case_01_Output.png
    └── ...
```

How to Run & Testing Guidelines

1. Install Python 3 and clone or download this repository.
2. Open the folder in PyCharm or Visual Studio Code.
3. Run any script: `python Case_01_University_Scholarship.py`
4. Enter console inputs and compare against test cases (normal, boundary, ineligible, and invalid values).

Required Deliverables per Case Study

Problem statement • Objectives • Assumptions • Algorithm • Flowchart • Pseudocode • Python source code • Test cases • Output screenshots • Conclusion.

Conclusion

This project demonstrates how Python programming can be used to solve practical problems in education, electricity billing, healthcare, payroll, telecommunications, railway reservations, retail, manufacturing, travel, and campus event planning. It helps develop problem-solving skills and builds a foundation for more advanced programming and AI & ML concepts.

Submitted by: _____ **Date:** _____